DBT PORTFOLIO PROJECT

LAYER: INTERMEDIATE

DBT Intermediate Model

Forecasting | Jinja2 | Compound Growth | SQL Generation | Snowflake

Project Summary

Generates a 12-period forward forecast per entity using compound growth -- base_value * (1 + r)^n -- where r is configurable via a Jinja2 set variable. Uses a Jinja2 for-loop to generate a UNION ALL SQL spine at compile time, eliminating the need for recursive CTEs or external scheduling. Demonstrates DBT's templating power applied to financial planning forecasts.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	int_straight_line_forecast
Layer:	Intermediate
Materialization:	view
Growth Rate:	5% per period (r=0.05, configurable via Jinja set variable)
Forecast Horizon:	12 periods ahead from each entity's most recent actual
Key Outputs:	periods_ahead, forecast_value, base_value, growth_rate_applied
Audience:	Finance/FP&A, Supply Chain Planning, Tableau planning dashboards

2 BUSINESS PROBLEM

Finance planning teams needed a quick forward projection for each entity's claim metric -- a 12-month outlook that could be refreshed automatically every time the dbt pipeline runs. Rather than maintaining a separate Python script or spreadsheet, this model generates the forecast directly in Snowflake using the most recent actual as the base, applying compound growth for each future period.

- Compound growth: forecast_value = base_value * (1 + r)^n -- models realistic exponential trajectory.
- Jinja2 for-loop: Compiles to 12 UNION ALL SELECT blocks at dbt run time -- no recursion needed.
- Configurable r: Change {% set r = 0.05 %} at the top to update all 12 periods instantly.
- recency_rank = 1: Anchors to the most recent actual value per entity as the forecast base.
- Auto-refreshing: Every dbt run re-anchors to the latest data -- no manual date updates.

3 TECHNICAL EXPLANATION

- actuals CTE: Pulls all rows; adds recency_rank (ROW_NUMBER DESC by period_date) to identify the most recent record.
- latest_actuals CTE: Filters recency_rank = 1 -- one row per entity with base_date and base_value.
- Jinja2 namespace + for-loop: {% set ns.period_list %} builds a list [1..12]. The for-loop then generates one SELECT block per period with {{ i }} substituted in at compile time.
- forecast_spine CTE: Each SELECT computes base_value * power(1 + r, i) for that period's horizon.
- UNION ALL stitches all 12 period blocks into a single result set without recursion.
- Final SELECT: Rounds forecast_value to 4 decimal places and appends growth_rate_applied for auditability.

Key Techniques: Jinja2 set/for/namespace, UNION ALL spine, POWER() compound growth, ROW_NUMBER() recency anchor

4 FULL MODEL CODE

```
{{ config(materialized='view') }}

{% set r = 0.05 %}
{% set forecast_periods = 12 %}

with actuals as (
    select
        entity_id,
        period_date,
        metric_value,
        max(period_date) over (partition by entity_id) as last_actual_date,
        row_number() over (
            partition by entity_id order by period_date desc
        ) as recency_rank
    from {{ source('healthcare', 'claims') }}
),

latest_actuals as (
    select
        entity_id,
        period_date as base_date,
        metric_value as base_value
    from actuals
    where recency_rank = 1
),

{% set ns = namespace(period_list=[]) %}
{% for i in range(1, forecast_periods + 1) %}
    {% set ns.period_list = ns.period_list + [i] %}
{% endfor %}

forecast_spine as (
    {% for i in ns.period_list %}
    select
        entity_id,
        base_date,
        base_value,
        {{ i }} as periods_ahead,
        base_value * power(1 + {{ r }}, {{ i }}) as forecast_value
    from latest_actuals
    {% if not loop.last %} union all {% endif %}
    {% endfor %}
)

select
    entity_id,
    base_date,
    base_value,
    periods_ahead,
    round(forecast_value, 4) as forecast_value,
    {{ r }} as growth_rate_applied
from forecast_spine
order by entity_id, periods_ahead
```

entity_id	base_date	base_value	periods_ahead	forecast_value	growth_rate
E-001	2025-12-01	4890.55	1	5135.08	0.05
E-001	2025-12-01	4890.55	3	5664.87	0.05
E-001	2025-12-01	4890.55	6	6556.50	0.05
E-001	2025-12-01	4890.55	12	8794.63	0.05

** Growth rate r=0.05 (5% per period). Change {% set r = 0.XX %} to retune all 12 periods instantly.*